sentinel-oo: API manual

Anil Wipat, Newcastle University

11 November 2026

Contents

sentinel-oo: API manual **1**
Which one to use . . . 2

Part A: the web interface **2**
The endpoints . . . 2
`POST /check/{contract}` . . . 2
`GET /contracts` . . . 3
`GET /contracts/{name}` . . . 4
`GET /runs` . . . 4
`GET /graph/{run}` . . . 5
`GET /health` . . . 5
Status codes and errors . . . 6

Part B: the agent interface **6**
Starting it . . . 7
Configuring a coordinator . . . 7
The protocol methods . . . 7
The four tools . . . 8
What the agent interface never does . . . 9
Errors . . . 9

Both interfaces agree **10**

Known limitations **10**

sentinel-oo: API manual

Two ways to call the service. Both are thin adapters over the same orchestrator, and **a test fails if they ever give different verdicts.**

	Who calls it	Gets the crate back?
The web interface	A pipeline, or a person	Yes
The agent interface	A coordinating agent	No, deliberately

Every response in this document was captured from the running service on 11 November 2026. None of it is a mock-up.

Which one to use

Use the web interface if the caller is a pipeline step, a script, or a person. The crate comes back in the reply, and the crate is the thing that carries on down the pipeline with the service's record inside it.

Use the agent interface if the caller is a coordinating agent in a multi-agent system. It speaks Model Context Protocol and never returns the crate, because a crate is thousands of lines of JSON: putting it in an agent's context costs money on every call and invites the agent to start editing laboratory records.

Part A: the web interface

Plain HTTP with JSON bodies. No authentication, no API key, no session. Built on Python's standard library, so the service runs with nothing installed.

Base address is wherever you deployed it, which `deploy.py` prints. The examples below use `http://127.0.0.1:8080`.

The endpoints

Method and path	What it does
`POST /check/{contract}`	The whole job. A crate in, a checked crate out
`GET /contracts`	Which contracts this instance holds
`GET /contracts/{name}`	One contract in full, and in plain words
`GET /runs`	The runs this instance is still holding
`GET /graph/{run}`	Not built. Returns 501
`GET /health`	The build, and which model if any
`GET /openapi.json`	A description of the service, generated from the route table

The route table is the single source of truth and the OpenAPI document is generated from it, so the description cannot drift away from the service. A test checks that.

`POST /check/{contract}`

The only endpoint that does anything. Send the contents of an `ro-crate-metadata.json` file as the body.

```
curl -s -X POST --data-binary @<a crate>/ro-crate-metadata.json \
  http://127.0.0.1:8080/check/design_to_build
```

The reply


```
{
  "run": "e0f8a2ac6fe3",
  "verdict": "REFUSED",
  "findings": [
    {
      "requirement": "everything_the_crate_lists_is_present",
      "severity": "refuse",
      "where": "./",
      "value": "sequencing-report.pdf",
      "message": "The crate lists a file as part of itself that is described nowhere in it.
    Whoever receives this will look for it and not find it."
```

```
    }
  ],
  "crate": { "@context": [ ... ], "@graph": [ ... ] }
}
```

Field	What it is
`run`	This run's identifier. Also returned in the `X-Sentinel-Run` header
`verdict`	`ACCEPTED`, `ACCEPTED_WITH_FLAGS` or `REFUSED`
`findings`	Everything the contract found, flags included, not only refusals
`crate`	The crate you sent, with the record of this check inside it

Each finding carries the requirement's name, its severity, the entity it was found on, the offending value where there is one, and the message **from the contract file**. That message is somebody's words in a YAML file, not the service's opinion.

The verdicts

Verdict	Meaning	What the caller should do
`ACCEPTED`	Nothing found	Proceed
`ACCEPTED_WITH_FLAGS`	Nothing stops it, something was recorded	Proceed, and read the flags
`REFUSED`	At least one requirement with severity `refuse` failed	Do not proceed

A refusal still returns the crate. A refusal that exists only as an error message in somebody's terminal is not a record.

What the caller must not do

Do not decide the verdict yourself by counting findings. `ACCEPTED_WITH_FLAGS` has findings and is not a refusal. Read `verdict`.

`GET /contracts`

```
{
  "contracts": [
    { "name": "build_to_test",   "version": "0.2.0",  "requirements": 2 },
    { "name": "design_to_build", "version": "0.10.0", "requirements": 18 }
  ],
  "checksum": "sha256:a23c50aa6e87a30d16703da6126a54b90dc5dde4ec5d82bc7d0a93719992b682"
}
```

The checksum covers all of them together. It is what `deploy.py` compares after starting an instance, to prove the running service holds exactly the contracts that were validated.

GET /contracts/{name}

Everything about one contract, so a producing team can check itself before being checked without reading any YAML.

```
{
  "name": "design_to_build",
  "version": "0.10.0",
  "checksum": "sha256:d38b1f0e...",
  "description": "What the Design phase must hand to the Build phase ...",
  "summary": "design_to_build version 0.10.0\n\n...",
  "expects": ["PlateWell", "IntendedPlateLayout", "ExpressionConstruct",
              "Protein", "GeneCallingStep"],
  "requirements": [ ... ]
}
```

`expects` lists the entity types the contract is written for. A crate containing none of them is refused before any requirement is applied.

`summary` is the whole contract in plain words. **That is the field to send to a producing team.**

One requirement

```
{
  "name": "edge_handling_is_recorded",
  "applies_to": "GeneCallingStep",
  "reads": "its named value 'edge handling'",
  "must": "be recorded",
  "severity": "refuse",
  "message": "The gene calling step does not say how it treated the edges of contigs, so nobody
     can tell whether any sequence in this panel may be a fragment.",
  "one_of": null,
  "of_type": null,
  "between": null,
  "repairable": true
}
```

Field	What it is
`applies_to`	`root`, `descriptor`, or an entity type
`reads`	The property, or the named value, this requirement looks at
`must`	One of four verbs
`severity`	`refuse` or `flag`
`message`	The sentence a person reads, from the contract file
`one_of`	The only permitted values, or null
`of_type`	What a reference must point at, or null
`between`	`[low, high]` for the numeric range verb, or null
`repairable`	Whether a model may be asked to recover this value

GET /runs

```
{ "runs": ["318d530ac52d", "d71ef1345972"], "held": 2, "limit": 20 }
```

Runs are held in memory and the oldest fall off. **This is a convenience, not a record.** The durable record is inside the crate that was returned.

GET /graph/{run}

Not built. It returns **501**, which in HTTP means the server does not support this.

```
{
  "error": "not built yet",
  "detail": "the working graph arrives with the graph store, at a later step of the build order
    . Nothing is served here rather than something that is not the graph."
}
```

The endpoint exists so the shape of the interface is settled. It says so rather than returning an empty graph that would look like an answer.

For a run the service is not holding, it returns **404** instead.

GET /health

```
{
  "alive": true,
  "service": "sentinel-oo",
  "version": "0.1.0",
  "image_tag": "dev",
  "git_commit": "c0525b0",
  "rules_checksum": "none",
  "model": {
    "state": "reachable",
    "engine": "qwen2.5:14b at http://model:11434"
  },
  "model_engine": "qwen2.5:14b at http://model:11434",
  "contracts": 2
}
```

The three model states

A model is called by default and runs in its own container, which does not have to be on this machine. `state` is one of three:

`state`	Meaning
`none configured`	No model was asked for. A deliberate state
`reachable`	A model is configured and answering
`configured but not answering`	A model was wanted and cannot be reached

The third carries two more fields, `detail` with the error and `effect` saying what follows:

```
{
  "state": "configured but not answering",
  "engine": "qwen2.5:14b at http://gpu-box.lan:11434",
  "detail": "URLError: <urlopen error [Errno 8] nodename nor servname provided>",
  "effect": "checks run normally and no verdict changes. A value recorded only in free text
    cannot be recovered."
}
```

It is still status 200 in all three cases, because being without a model is a deliberate state and not a failure, and no verdict depends on it.

Before 11 November 2026 all three looked identical, which is how an instance can appear to be working while doing none of what it was deployed to do.

`model_engine` is the same name as a plain string, kept so that anything already reading that field still works.

Status codes and errors

Every error is JSON with an `error` field and usually a `detail` that says what to do.

Code	When	Example body
200	It worked, including a REFUSED verdict	
400	The body is not a crate	`{"error": "not a crate", "detail": "no @graph. This is JSON but not an RO-Crate.", "hint": "..."}`
404	No such contract	`{"error": "no such contract", "detail": "no contract called 'nosuch'. This service knows: build_to_test, design_to_build."}`
404	No such path	`{"error": "no such path", "known": ["/check/{contract}", "/contracts", ...]}`
404	No such run	`{"error": "no such run", "detail": "this service no longer holds that run, or never did. The durable record is in the crate that was returned."}`
405	Wrong method for that path	`{"error": "wrong method", "detail": "/health answers GET, not PUT"}`
500	The service failed while answering	`{"error": "the service failed while answering", "detail": "..."}`
501	`/graph`, which is not built	

A REFUSED verdict is status 200. The request succeeded; the handover did not. Do not treat a refusal as an HTTP error.

The body is checked before the contract name. Sending a malformed body to an unknown contract reports the body, because that is the first thing it tried to do.

`GET`, `POST`, `PUT`, `PATCH` and `DELETE` all reach the dispatcher, so every one of them answers with JSON. Before 11 November 2026 the last three did not, and Python's base handler answered them with a page of HTML.

Part B: the agent interface

Model Context Protocol, spoken as JSON-RPC 2.0 over standard input and output, one JSON object per line. Protocol version `2025-06-18`.

Starting it

```
docker run -i --rm \
  -v "$PWD/<your contracts>:/contracts:ro" \
  -v "$PWD/<your crates>:/crates:ro" \
  sentinel-oo:dev --agent-interface
```

A coordinating agent starts this itself. It is not a long-running server and there is no port.

Configuring a coordinator

Most agent frameworks take a command and its arguments:

```
{
  "mcpServers": {
    "sentinel-oo": {
      "command": "docker",
      "args": [
        "run", "-i", "--rm",
        "-v", "/path/to/contracts:/contracts:ro",
        "-v", "/path/to/crates:/crates:ro",
        "sentinel-oo:dev", "--agent-interface"
      ]
    }
  }
}
```

Not yet tested against a real Model Context Protocol client. It is tested against the protocol as written down, and every exchange in this document was captured from the running container by hand.

The protocol methods

Method	What it does
`initialize`	Returns the protocol version and server identity
`tools/list`	The four tools and their input schemas
`tools/call`	Run one tool
`ping`	Returns an empty result
`notifications/initialized`	Accepted, no reply, as the protocol requires

A request with no `id` is a notification and gets no answer.

`initialize`

```
{"jsonrpc":"2.0","id":1,"method":"initialize","params":{}}
```

```
{
  "protocolVersion": "2025-06-18",
  "capabilities": { "tools": {} },
  "serverInfo": { "name": "sentinel-oo", "version": "0.1.0" }
}
```

The four tools

`check_handover`

The one that does the work.

Argument	Required	What it is
`handover`	yes	Which contract to check against, by name
`crate_path`	yes	Path to an `ro-crate-metadata.json`, or the folder holding it
`out_path`	no	Where to write the checked package

Without `out_path` a writable folder is chosen and the reply says which, in `checked_package_written_to`.

```
{"jsonrpc":"2.0","id":2,"method":"tools/call","params":{
  "name":"check_handover",
  "arguments":{"handover":"design_to_build",
               "crate_path":"/crates/design-to-build/invalid/mode-in-prose"}}}
```

The result's `content[0].text` holds JSON:

```
{
  "run": "f3ff8922b842",
  "verdict": "REFUSED",
  "contract": "design_to_build",
  "contract_version": "0.10.0",
  "findings": [
    {
      "requirement": "edge_handling_is_recorded",
      "severity": "refuse",
      "where": "#act-gene-calling",
      "value": null,
      "message_from_contract": "The gene calling step does not say how it treated the edges of
    contigs, so nobody can tell whether any sequence in this panel may be a fragment."
    }
  ],
  "checked_package_written_to": "/tmp/sentinel-oo/checked-f3ff8922b842/ro-crate-metadata.json",
  "note": "The package itself is not returned here because it is large enough to be unhelpful
    in a conversation. Read it from the path above if something needs it."
}
```

The findings field is called `message_from_contract` and not `message`. That is deliberate. An agent reading it can tell the wording came from a contract file somebody wrote, and is not an instruction from this service.

`list_handovers`

No arguments.

```
{
  "handovers": [
    { "name": "build_to_test",   "version": "0.2.0",  "requirements": 2 },
    { "name": "design_to_build", "version": "0.10.0", "requirements": 18 }
  ]
}
```

`read_contract`

Takes `handover`. Returns the contract in plain words, under a field named `requirements_in_plain_words_from_the_contract`.

Use this to tell a producing phase what it must supply before it is checked.

`get_run_graph`

Takes `run`. Not built, and says so.

What the agent interface never does

It never returns the crate. Only the verdict, the findings and the path the checked package was written to.

Its tool descriptions never come from contract files. They are fixed in code. Without that rule, whoever writes a contract is writing instructions for the agent, and this service accepts contracts from the sender.

It never decides anything. Like the web interface, it is a thin adapter over the same orchestrator.

Errors

A tool that fails returns `isError: true`, with the detail in the text. **Nothing a tool does can take the service down.**

A wrong argument name

The caller is a model, and models get argument names wrong.

```
{
  "error": "wrong arguments",
  "detail": "AgentInterface.check_handover() got an unexpected keyword argument 'out_dir'",
  "expected": ["crate_path", "handover", "out_path"],
  "required": ["handover", "crate_path"]
}
```

It names what the tool does take, which turns a dead end into one more attempt. Before 11 November 2026 this killed the process: the agent got no answer at all, not even an error, and the service was gone.

Other errors

`error`	When
`no such tool`	With `known`, listing the four
`no such handover`	With the names it does know
`not a crate`	The path is not an RO-Crate
`the checked package could not be written`	With a hint, and the verdict, since the crate was checked and only the saving failed
`the tool failed`	Anything else, with the exception type and message

A JSON-RPC error with code `-32601` means an unknown method, `-32700` means the line was not JSON, and `-32603` means a fault above the tools, which is answered rather than crashing.

Both interfaces agree

`test_both_interfaces_give_the_same_verdict` runs the same crates down both paths and compares the verdict and the list of findings. If a change ever lets the web interface accept something the agent interface refuses, that test fails.

Known limitations

`GET /graph/{run}` is not built, and returns 501 rather than pretending.

The agent interface has not been tested against a real Model Context Protocol client.

There is no authentication of any kind. Deploy it where only the things that should call it can reach it.

Runs are held in memory and the oldest fall off. The durable record is inside the returned crate.

No streaming and no batching. One crate per request.